

1. **Scoping styles:** Why global CSS collides in a component app · One CSS file per component, and BEM names · CSS Modules: class names scoped to one file · Conditional classes: the state picks the class · Inline styles: `style={{ }}` for values computed at runtime
2. **Approaches, themes and layout:** Utility-first CSS: Tailwind · CSS-in-JS: styles written in JavaScript · Choosing one styling approach · Design tokens and themes with CSS custom properties · A responsive component: breakpoints in its own CSS · Hover, focus and disabled: interaction states
3. **Tests and tooling:** Why automated tests · The testing pyramid: unit, component, end-to-end · Vitest: describe, it, expect · Tooling cheat-sheet: ESLint, Prettier, React DevTools
4. **Finding elements:** Testing Library: test what the user sees · Finding elements: by role, label and text · `getBy`, `queryBy`, `findBy`: present, absent, later
5. **Interactions and async UI:** Simulating the user: `userEvent` and mock callbacks · Testing async UI: waiting and mocking fetch · What not to test: implementation details · A first component test, end to end · Components that need a provider or a router: render with a wrapper

Scoping styles

Why global CSS collides in a component app

In a component app every imported CSS file joins **one global stylesheet**, so a class such as `.title` from one component also styles every other `.title` on the page.



Two files, one stylesheet: the later `.title` rule wins for both cards.

- **Order follows imports:** which file loads last depends on the import order, so a style can change because someone added an import in a file you never opened. The cause is far from the symptom.
- **Generic names collide most:** `.button`, `.card`, `.active`, `.error`, `.title`.

Every approach in this section gives each class a unique name

BEM A naming convention you keep by hand.

CSS Modules The build tool renames each class per file.

Utility classes No component names to invent at all.

CSS-in-JS A library generates the names.

```

function TaskCard() {
  return (
    <article className="card">
      <h2 className="title">Buy milk</h2>
      <p>Due today</p>
    </article>
  );
}

function ProfileCard() {
  return (
    <article className="card">
      <h2 className="title">Ana Ruiz</h2>
      <p>Admin</p>
    </article>
  );
}
  
```

```
export default function App() {
  return (
    <main>
      <TaskCard />
      <ProfileCard />
    </main>
  );
}
```

Example. In the Try it box, the author of TaskCard wrote an orange title, yet both titles are blue and uppercase. Both `.title` rules score (0, 1, 0), so the one later in the stylesheet wins, and it applies to **every** `.title`, not just the profile card. Delete the second rule: now both titles are orange. Rename the classes to `task-card__title` and `profile-card__title` (in the JSX and the CSS) and each card keeps its own look, whatever the order.

Common mistake. Fixing a collision by making the selector heavier: `.task-list .card .title`, or `!important`. It wins today and starts a specificity war tomorrow, when someone needs to override it. A collision is a **naming** problem: fix the name, not the weight.

One CSS file per component, and BEM names

The simplest discipline: each component gets its own CSS file next to it, and every class in it starts with the component's name, following a convention such as **BEM**: `.task-card`, `.task-card__title`, `.task-card--done`.

Block: `task-card` The standalone component.
Element: `task-card__title` A part that only makes sense inside the block, joined with two underscores.
Modifier: `task-card--done` A variant or a state, joined with two hyphens (also `task-card__title--large`).

Where the files go

- **Side by side:** `TaskCard.jsx` and `TaskCard.css`, the CSS imported once at the top of the component. Deleting the component deletes its styles too.
- **One global file:** what is truly global (a reset, box-sizing, the body font, [design tokens](#)) goes in `index.css`, imported once in `main.jsx`.

Folder tree

```
src/
components/
  TaskCard.jsx
  /* only .task-card... classes */
  TaskCard.css
  ProfileCard.jsx
  /* only .profile-card... classes */
  ProfileCard.css
/* reset, fonts, tokens: imported once in main.jsx */
index.css
```

```
import { useState } from 'react';
import './TaskCard.css';

function TaskCard({ title }) {
  const [done, setDone] = useState(false);
  return (
    <article className={done ? 'task-card task-card--done' : 'task-card'}>
      <h2 className="task-card__title">{title}</h2>
      <button className="task-card__button" onClick={() => setDone(!done)}>
        {done ? 'Undo' : 'Done'}
      </button>
    </article>
  );
}

export default function App() {
```

```

return (
  <>
    <TaskCard title="Buy milk" />
    <TaskCard title="Call Ana" />
  </>
);
}

```

Example. In the Try it box, press **Done** on the first card. Only that `<article>` gets the modifier class `task-card--done`; the CSS for the modifier strikes through the title inside it. The second card is untouched because the modifier is on its own block. The class names tell you, in the DevTools Elements panel, exactly which component and which part you are looking at.

Common mistake. Believing the import scopes the CSS: "these rules are in `TaskCard.css`, so they only affect `TaskCard`". Once loaded, they are global, and they stay on the page even when no `TaskCard` is rendered. A rule like `.task-card h2 { ... }` is safe only because of the `task-card` prefix; a bare `h2 { ... }` in that file restyles every heading in the app.

CSS Modules: class names scoped to one file

A **CSS Module** is a CSS file named `*.module.css`: the build tool renames every class in it to a unique name and hands the component an object that maps your names to the real ones, `styles.title` → `"_title_1hy3p_5"`.



One short name in, one unique name out, shared by the CSS and the JSX.

- **Read names from the object:** import styles from `'./TaskCard.module.css'`, then `className={styles.title}`.
- **Several classes** are joined into one string: `[styles.card, styles.done].join(' ')` (see [Conditional classes](#)).
- **Dashes need brackets**, `styles['is-done']`, which is why many teams write camelCase in modules: `.isDone` → `styles.isDone`.

What is not renamed, and two extras

- **Only classes** (and ids and animation names) are renamed. A tag selector such as `h2 { ... }` in a module is still global, so keep module selectors class-based.
- **composes:** `base`; inside a rule reuses another class of the same file, or of another one: `composes: card from './Card.module.css'`.
- `:global(.dark) .card { ... }` refers on purpose to a global class, for example a theme class on `<html>`.

```

import { useState } from 'react';
import styles from './TaskCard.module.css';

function TaskCard({ title }) {
  const [done, setDone] = useState(false);
  const cardClass = done ? [styles.card, styles.done].join(' ') : styles.card;
  return (
    <article className={cardClass}>
      <h2 className={styles.title}>{title}</h2>
      <button className={styles.button} onClick={() => setDone(!done)}>
        {done ? 'Undo' : 'Done'}
      </button>
    </article>
  );
}

```

```
export default function App() {
  return (
    <>
      <TaskCard title="Write the report" />
      <TaskCard title="Book the dentist" />
    </>
  );
}
```

Example. In a Vite app, open the Elements panel after rendering this card: you see `<h2 class="_title_1hy3p_5">`, not `title`. Another component can declare its own `.title` in its own module and both keep their colours, in any import order. In the Try it box, press **Done** and watch the class list of the `<article>` become `card done`.

Common mistake. Writing the class as a string after importing the module: `className="title"`. The real class is `_title_1hy3p_5`, so the string matches no rule and the styles silently vanish (no error). Always read the name from the object, `className={styles.title}`. A misspelled property, `styles.titel`, is undefined, and fails just as silently.

Conditional classes: the state picks the class

To change how a component looks when its state changes, compute the `className` from the state; the CSS describes each look and React only switches between them.

Three ways to build the class string

Case	Code
Two cases	<code>className={done ? 'task done' : 'task'}</code>
Several optional classes	<code>[styles.task, done && styles.done, urgent && styles.urgent].filter(Boolean).join(' '): filter(Boolean)</code> drops the false entries
The <code>clsx</code> package	<code>clsx(styles.task, { [styles.done]: done })</code>

```
import { useState } from 'react';
import styles from './Task.module.css';

function Task({ title, urgent }) {
  const [done, setDone] = useState(false);
  const className = [styles.task, done && styles.done, urgent && styles.urgent]
    .filter(Boolean)
    .join(' ');
  return (
    <li className={className}>
      <span>{title}</span>
      <button aria-pressed={done} onClick={() => setDone(!done)}>Done</button>
    </li>
  );
}

export default function App() {
  return (
    <ul className={styles.list}>
      <Task title="Pay the rent" urgent />
      <Task title="Water the plants" />
    </ul>
  );
}
```

Example. In the Try it box, "Pay the rent" starts with `task urgent` (red border). Press its **Done**: the class list becomes `task done urgent` and the button gets `aria-pressed="true"`, which the CSS uses to fill it in. The same attribute tells a screen-reader user that the button is now pressed.

Common mistake. Gluing a condition into a template literal, such as `task ${done && 'done'}` between backticks. When `done` is `false` the result is `"task false"` (and `"task undefined"` for a missing prop): a junk class that looks harmless until someone names a class `false`. Use a ternary that returns `' '`, or `filter(Boolean)`.

Inline styles: `style={{ }}` for values computed at runtime

The `style` prop takes a JavaScript object with camelCase properties, `style={{ width: '40%' }}`: the right tool for values that come from data, and classes for everything else.

- **camelCase names:** `backgroundColor`, `marginTop` (the [JSX rules](#)).
- **Values** are strings (`'40%'`, `'#1a1f6c'`) or numbers.
- **Numbers** get `px` for length properties (`marginTop: 8` is `8px`), but not for unitless ones (`lineHeight: 1.5`, `opacity: 0.5`, `zIndex: 2`).

```
import { useState } from 'react';

export default function App() {
  const [done, setDone] = useState(3);
  const total = 8;
  const percent = Math.round((done / total) * 100) + '%';
  return (
    <section>
      <label>
        Tasks done: {done} of {total}{' '}
        <input type="range" min="0" max={total} value={done}
          onChange={(e) => setDone(Number(e.target.value))} />
      </label>
      <p>1. Inline width:</p>
      <div className="track">
        <div className="fill" style={{ width: percent }} />
      </div>
      <p>2. A custom property, the rule stays in CSS:</p>
      <div className="track meter" style={{ '--progress': percent }} />
    </section>
  );
}
```

Example. Move the slider. Both bars follow the data, but only the second one animates (the `transition` is in CSS) and changes colour on hover (`:hover` cannot be written inline). The component passes **one number**; the look stays in the stylesheet.

Common mistake. Writing the style as an HTML string: `style="width: 40%"`. React throws "The `style` prop expects a mapping from style properties to values, not a string". The prop is an object: `style={{ width: '40%' }}`, with the outer braces opening JavaScript and the inner ones creating the object.

Approaches, themes and layout

Utility-first CSS: Tailwind

In **utility-first CSS** you combine many tiny classes, each setting one property, directly in the markup (`className="flex items-center gap-2 p-4"`); **Tailwind CSS** is the best-known framework of this kind.

- **The spacing scale:** one step is `0.25rem`, so `p-4` is `1rem` of padding and `gap-2` is `0.5rem`.
- **Variants** are prefixes that apply a class under a condition: `hover:bg-blue-700`, `focus-visible:outline-2`, `disabled:opacity-50`, `dark:bg-slate-900`.
- **Breakpoints** are mobile-first prefixes: `md:grid-cols-2` applies from the `md` width (`48rem`) upwards, like a `min-width` media query.

Pros No names to invent, no collisions, no dead CSS left behind, one spacing and colour scale, and the styles are visible where you read the markup.

Cons Long class strings, a vocabulary to learn, and every visual change is a markup change. Repetition is solved with components, not new CSS: if twelve buttons share twelve classes, write one `<Button>`.

Set-up (Tailwind v4 with Vite)

1. `npm install tailwindcss @tailwindcss/vite`
2. Add `tailwindcss()` to the plugins of `vite.config.js`.
3. Put `@import "tailwindcss";` in your main CSS file.

Common Tailwind utilities and the CSS they stand for

Class	CSS
<code>flex · grid · hidden</code>	<code>display: flex · display: grid · display: none</code>
<code>items-center · justify-between</code>	<code>align-items: center · justify-content: space-between</code>
<code>gap-2 · p-4 · px-3 py-1 · mt-2</code>	<code>gap: .5rem · padding: 1rem · padding left/right .75rem, top/bottom .25rem · margin-top: .5rem</code>
<code>text-sm · font-bold · text-white</code>	<code>font-size: .875rem · font-weight: 700 · color: white</code>
<code>bg-blue-600 · rounded-lg · shadow</code>	<code>a blue background from the palette · border-radius: .5rem · a box shadow</code>
<code>w-full · max-w-md</code>	<code>width: 100% · max-width: 28rem</code>
<code>hover:... · focus-visible:... · md:... · dark:...</code>	<code>the class applies on hover · on keyboard focus · from 48rem wide · in dark mode</code>

```
// With Tailwind installed: no CSS file for this component
function TaskCard({ title, done, onToggle }) {
  return (
    <article className="flex items-center justify-between gap-2 rounded-lg border p-4">
      <h2 className={done ? 'text-gray-500 line-through' : 'font-bold'}>{title}</h2>
      <button
        onClick={onToggle}
        className="rounded bg-blue-600 px-3 py-1 text-white hover:bg-blue-700 focus-visible:outline-2"
      >
        {done ? 'Undo' : 'Done'}
      </button>
    </article>
  );
}
```

```
// A few utilities written by hand (see the CSS pane): Tailwind generates
// thousands of them, but each one is this simple.
function TaskCard({ title }) {
  return (
    <article className="flex items-center justify-between gap-2 p-4 rounded-lg border">
      <h2 className="text-sm font-bold">{title}</h2>
      <button className="px-3 py-1 rounded bg-blue-600 text-white">Done</button>
    </article>
  );
}

export default function App() {
  return <TaskCard title="Buy milk" />;
}
```

Example. In the Try it box, change `p-4` to `p-1` in the JSX: the card shrinks, without touching the CSS. That is the workflow with utilities: you edit the markup, choosing from a scale, and never write a new rule for one component.

Common mistake. Building class names at runtime, such as the template literal `bg-${color}-600`. Tailwind finds classes by scanning the **source text** for complete names; `bg-red-600` never appears whole in the file, so it is never generated and the style silently does not exist. Write the full names and pick one: `{ red: 'bg-red-600', blue: 'bg-blue-600' }[color]`.

CSS-in-JS: styles written in JavaScript

CSS-in-JS libraries such as **styled-components** let you write real CSS inside JavaScript and get back a component with those styles attached, under a generated class name.

The price: work in the browser

- **Extra JavaScript** to download.
- **Work during rendering:** the library parses CSS, computes class names and inserts rules while the app runs, again whenever a prop changes the CSS.
- **Styles wait for the JavaScript:** they cannot arrive before it.

Runtime CSS-in-JS styled-components: CSS produced in the browser.

Zero-runtime CSS-in-JS vanilla-extract, Linaria, Panda CSS: styles written next to the component in JavaScript or TypeScript, extracted as plain CSS files at build time, like CSS Modules.

```
import styled from 'styled-components';

// A component with its CSS. $primary is a "transient" prop:
// the $ keeps it from being passed on to the <button> element.
const Button = styled.button`
  padding: 6px 14px;
  border-radius: 6px;
  border: 2px solid navy;
  background: ${props => (props.$primary ? 'navy' : 'white')};
  color: ${props => (props.$primary ? 'white' : 'navy')};

  &:hover { opacity: 0.85; }
`;

export function Actions() {
  return (
    <>
      <Button $primary>Save</Button>
      <Button>Cancel</Button>
    </>
  );
}
```

Example. Rendering `Actions` produces two `<button>` elements with two different generated classes (one for each value of `$primary`) and one `<style>` tag the library added to the `<head>`. Nothing in the project is called `.button`, so nothing can collide with it.

Common mistake. Defining a styled component **inside** another component's body. Every render then creates a new component type, so React throws away the old `<button>` and mounts a new one (it loses focus and any state below it), and the library generates fresh class names each time. Define styled components at the top level of the module, like any component.

Choosing one styling approach

Every styling approach solves the same problem, **scoping**: pick **one** per project, add one global file for the reset and the design tokens, and keep inline styles for values computed at runtime.

The global layer every project needs

- **A reset:** `box-sizing: border-box`.
- **The body font.**
- **The design tokens** as custom properties.
- **A visible focus style.**

Styling approaches compared

Approach	Scoping	Dynamic styles	Runtime cost	Good fit
----------	---------	----------------	--------------	----------

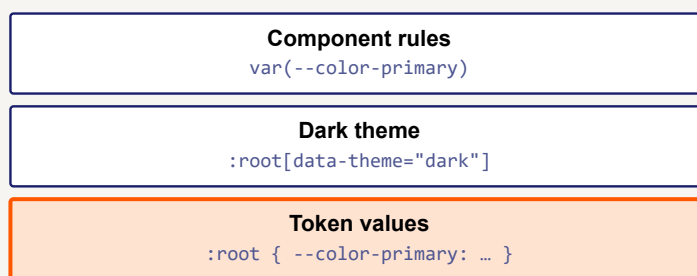
Approach	Scoping	Dynamic styles	Runtime cost	Good fit
Plain CSS + BEM	by convention (discipline)	classes, attributes, custom properties	none	small apps; teams fluent in CSS
CSS Modules	automatic, per file	the same, plus composes	none (build time)	the smallest step up from plain CSS
Utility classes (Tailwind)	no names to collide	conditional class strings, variants	none (build time)	teams that want one design scale
Runtime CSS-in-JS (styled-components)	automatic	props inside the CSS	yes: work in the browser	existing codebases that use it
Inline <code>style={{ }}</code>	one element	any value	small	values that come from data, only

Example. A task manager styled with CSS Modules: `index.css` holds the reset, the tokens (`--color-primary`, `--space`) and the focus ring; every component has its own `X.module.css` using `var(--...)`; the progress bar passes its percentage through `style={{ '--progress': ... }}`. A newcomer finds any style in at most two places.

Common mistake. Mixing approaches: half the components in Tailwind, some in CSS Modules, a few in styled-components, colours repeated as hexes in each. Every change starts with "how is this one styled?", and the same blue drifts into five slightly different blues. If you must migrate, do it component by component, with shared tokens.

Design tokens and themes with CSS custom properties

A **design token** is a named design decision (the primary colour, the spacing step, the corner radius), stored as a **CSS custom property** such as `--color-primary` and read with `var(--color-primary)`.



Components use names; a theme only changes the values.

- **Define them on** `:root` (the `<html>` element) so everything inherits them; read them with `var(--name)`, optionally with a fallback: `var(--radius, 6px)`.
- **Set one from React:** `style={{ '--accent': color }}`; the CSS below that element sees the new value.

Switching themes

- **Follow the operating system:** `@media (prefers-color-scheme: dark) { :root { ... } }`.
- **Let the user choose:** `:root[data-theme="dark"] { ... }`, set from React with `document.documentElement.dataset.theme = theme` and remembered with [useLocalStorage](#).
- **Form controls too:** add `color-scheme: light dark` so inputs and scrollbars follow.

```
import { useEffect, useState } from 'react';

function ThemeToggle() {
  const [theme, setTheme] = useState('light');
  useEffect(() => {
    document.documentElement.dataset.theme = theme;
  }, [theme]);
  return (
    <button className="button" aria-pressed={theme === 'dark'}
      onClick={() => setTheme(theme === 'dark' ? 'light' : 'dark')}>
      Dark theme
    </button>
  );
}
```

```

    );
  }

  export default function App() {
    return (
      <main className="card">
        <h1 className="card__title">Tasks</h1>
        <p>Every colour and size on this card is a token.</p>
        <ThemeToggle />
      </main>
    );
  }

```

Example. In the Try it box, press **Dark theme**. The effect sets `data-theme="dark"` on `<html>`, the second `:root` rule redefines five tokens, and every rule that uses them repaints: no component rule was duplicated. Change `--radius` to `0` in the first block: the card and the button both become square.

Common mistake. Writing dark mode as a second copy of every component rule: `.dark .card { background: ... }`, `.dark .button { ... }`, and so on for each component. The copies drift apart and every new component needs two sets of rules. Redefine the **tokens** once per theme; components keep using `var(--...)`.

A responsive component: breakpoints in its own CSS

A component can carry its own breakpoint: a **media query**, or a **container query**, in the component's CSS changes its layout above a width, mobile-first.

Media query or container query

	Media query	Container query
Asks about	The viewport	The space the component actually gets
Set-up	None	<code>container-type: inline-size</code> on a wrapper
Syntax	<code>@media (min-width: 48rem) { ... }</code>	<code>@container (min-width: 30rem) { ... }</code>
Best for	The page layout	Reusable components that may sit in a page or a narrow sidebar

```

const tasks = ['Buy milk', 'Call Ana', 'Write the report', 'Book the dentist'];

function TaskGrid() {
  return (
    <section className="task-grid" aria-label="Tasks">
      <ul className="task-grid__list">
        {tasks.map((t) => <li key={t} className="task-grid__item">{t}</li>)}
      </ul>
    </section>
  );
}

// Drag the bottom-right corner of the dashed box to resize it.
export default function App() {
  return (
    <div className="resizable">
      <TaskGrid />
    </div>
  );
}

```

Example. In the Try it box, drag the corner of the dashed box: below 320px the tasks stack in one column, from 320px they form two, from 560px four. The window did not change size, only the container did; a media query could not have reacted to that.

Common mistake. Giving a component a fixed width, `.task-grid { width: 800px; }`, and testing only on a laptop. On a 375px phone it overflows and the page scrolls sideways. Let components fill the space they get (`max-width` instead of `width`) and add columns with a breakpoint.

Hover, focus and disabled: interaction states

Every interactive component needs a visible style for each state: hover, keyboard focus, disabled and "selected", the last one best expressed with attributes such as `[aria-pressed="true"]`.

:hover	The pointer is over the element.
:focus-visible	The browser decides the focus should be shown, typically when it came from the keyboard and not from a click. Give the ring contrast against the background (at least 3:1) and take its colour from a token.
:disabled	A control that cannot be used: make it look unavailable.
Selected states	Style the attributes the component already sets for accessibility: <code>a[aria-current="page"]</code> for the current link (router libraries set it on the active link), <code>button[aria-pressed="true"]</code> for a toggle, <code>[aria-invalid="true"]</code> for a field with an error.

```
import { useState } from 'react';

const pages = ['Tasks', 'Calendar', 'Settings'];

export default function App() {
  const [page, setPage] = useState('Tasks');
  return (
    <nav aria-label="Main" className="nav">
      {pages.map((p) => (
        <a key={p} href={`#${p.toLowerCase()}`} className="nav__link"
          aria-current={p === page ? 'page' : undefined}
          onClick={(e) => { e.preventDefault(); setPage(p); }}>
            {p}
          </a>
        )))}
      <button className="nav__button" disabled>Log out</button>
    </nav>
  );
}
```

Example. In the Try it box, click inside the preview and press Tab: each link shows the orange ring, but clicking with the mouse does not. Choose **Calendar**: React moves `aria-current="page"` to it, the CSS fills it in, and a screen reader announces "current page". The disabled **Log out** is skipped by Tab and looks unavailable.

Common mistake. Removing focus rings because they look "ugly": `*:focus { outline: none; }`. Keyboard users can no longer see where they are, which fails accessibility guidelines. Style `:focus-visible` the way you want instead; mouse users will not see it.

Tests and tooling

Why automated tests

An **automated test** is code that runs your code and checks the result, so everything can be checked again in seconds after every change.

Regressions caught at once	While the change is still fresh in your head.
Safe refactoring	Refactoring changes how code works inside without changing what it does; the tests confirm the outside still behaves.
Documentation	Test names state the intended behaviour: "disables Reset at zero".
Better design	Code that is hard to test is often doing too much.

```
// tasks.js
export function remaining(tasks) {
  return tasks.filter((t) => !t.done).length;
}
```

```
// tasks.test.js
import { describe, it, expect } from 'vitest';
import { remaining } from './tasks';

describe('remaining', () => {
  it('counts the tasks that are not done', () => {
    const tasks = [{ done: true }, { done: false }, { done: false }];
    expect(remaining(tasks)).toBe(2);
  });

  it('is 0 for an empty list', () => {
    expect(remaining([])).toBe(0);
  });
});
```

Example. Weeks later someone "simplifies" `remaining` and gets the filter backwards: `tasks.filter((t) => t.done)`. In watch mode the first test turns red at once, expected 1 to be 2, before the change ever reaches a user. Without the test, the counter on the screen would just be wrong until someone noticed.

Common mistake. Trusting a test you have never seen fail. A test with no `expect`, or one that checks a value the test itself set, passes whatever the code does. Make each new test fail once on purpose (break the code or the expected value) to prove it checks something; the test runner challenges do this for you with buggy components.

The testing pyramid: unit, component, end-to-end

Tests come in sizes, **unit**, **component** and **end-to-end**, and the pyramid says: many small, fast tests at the bottom, a few broad, slow ones at the top.

- **jsdom:** component tests run in jsdom, a JavaScript implementation of the DOM that runs in Node without a real browser. It is fast and handles elements, events and attributes, but does no layout or real rendering: it cannot tell you a button is off-screen or the wrong colour.
- **The trophy:** many front-end teams put most of their effort in component tests, because they resemble real use while staying fast.
- **A healthy mix:** unit tests for logic (validation, formatting, reducers), component tests for each important behaviour of a screen, a handful of end-to-end tests for the critical flows (log in, create a task).

Three levels of tests for a React app

Level	What it covers	Tools	Speed	When it fails, you learn
Unit	one pure function: <code>validateTask</code> , <code>formatDue</code> , a reducer	Vitest	milliseconds	exactly which function is wrong
Component	a component and its children in jsdom, user events, a mocked network	Vitest + React Testing Library	tens of milliseconds	which behaviour of which component broke
End-to-end	the real app in a real browser, with the real API	Playwright, Cypress	seconds per test	that a user flow broke, somewhere

Example. For a task app: a **unit** test checks that `validateTask({ title: '' })` returns "Title is required"; a **component** test checks that `TodoForm` shows that message and does not call `onAdd`; an **E2E** test logs in, creates a task, reloads the page and sees the task still in the list.

Common mistake. Testing everything end-to-end because "it is closest to reality". The suite takes twenty minutes, fails now and then for timing reasons, and a red result only says "something in this flow broke". People stop running it. Put each check at the lowest level that can catch the bug.

Vitest: describe, it, expect

Vitest is the test runner made for Vite projects: it finds the `*.test.js` and `*.test.jsx` files, runs them and reports each test as passed or failed.

Set it up and run it

1. Install it: `npm install -D vitest`.

2. Add "test": "vitest" to the scripts of package.json.

3. Run `npm test` (or `npx vitest`): **watch mode**, which reruns the affected tests on every save. `npx vitest run` runs once, as a CI server does.

<code>describe('TaskList', () => { ... })</code>	Groups related tests.
<code>it('shows the empty message', () => { ... })</code>	One test, named as a sentence about behaviour (test is the same).
<code>beforeEach(() => { ... })</code>	Runs before every test of the group, so each one starts clean.
<code>expect(value).matcher(expected)</code>	Checks a value; a failure shows the test name, the expected and received values and the line.

Matchers you will use most

Matcher	Passes when
<code>toBe(x)</code>	The same value (<code>Object.is</code>): strings, numbers, booleans
<code>toEqual(x)</code>	The same contents: objects and arrays
<code>toContain</code> , <code>toHaveLength</code>	An item is in the array or string; it has that length
<code>toBeNull</code> , <code>toBeTruthy</code> , <code>toThrow</code>	The value is null; truthy; the function throws
<code>.not...</code>	Negates any matcher
<code>toHaveBeenCalledWith(...)</code>	A mock function was called with these arguments: <code>vi.fn()</code> creates one, <code>vi.spyOn(object, 'method')</code> wraps an existing one

```
import { describe, it, expect, vi } from 'vitest';
import { addTask } from './tasks';

describe('addTask', () => {
  it('returns a new array with the task at the end', () => {
    const before = [{ id: 1, title: 'Buy milk' }];
    const after = addTask(before, { id: 2, title: 'Call Ana' });
    expect(after).toHaveLength(2);
    expect(after[1]).toEqual({ id: 2, title: 'Call Ana' }); // contents
    // a new array
    expect(after).not.toBe(before);
  });

  it('calls the logger once', () => {
    const log = vi.fn();
    addTask([], { id: 1, title: 'x' }, log);
    expect(log).toHaveBeenCalledTimes(1);
  });
});
```

Example. `npx vitest` prints ✓ `src/tasks.test.js` (2 tests). Change `toEqual` to `toBe` in the first test and it fails with expected `{ id: 2, title: 'Call Ana' }` to be `{ id: 2, title: 'Call Ana' }` // `Object.is` equality: the two objects look the same but are **different objects**, so they are not `Object.is-equal`.

Common mistake. Forgetting that a test of asynchronous code must wait: `it('loads', () => { loadTasks().then((t) => expect(t).toHaveLength(5)); })`. The test function returns before the promise settles, so it passes even when `loadTasks` is broken. Make the test `async` and `await` the call (or return the promise).

Tooling cheat-sheet: ESLint, Prettier, React DevTools

Three tools catch problems before tests do: **ESLint** finds bugs and broken hook rules, **Prettier** formats the code, and **React DevTools** shows the component tree, props and state.

- **ESLint's react-hooks rules** ship with Vite's React template: **rules-of-hooks** (no hooks in conditions or loops) and **exhaustive-deps** (every value an effect uses is in its dependency array, which prevents stale closures).

- **Testing plugins:** `eslint-plugin-testing-library` and `eslint-plugin-jest-dom` flag the patterns of What not to test; the "Advice" panel of the test runner here imitates a few of their rules.
- **React DevTools** (a browser extension): the Components tab shows the tree and lets you edit a component's props, state and hooks live; "Highlight updates when components render" flashes every re-render; the Profiler records what rendered and for how long.
- **The Elements tab** still matters: it shows the real class names (the hashed ones of CSS Modules) and the computed styles.

Tooling cheat-sheet

Tool	Catches	Run it	Tip
ESLint + react-hooks	bugs, unused code, broken hook rules, missing effect dependencies	<code>npm run lint (eslint .);</code> editor extension	fix warnings; do not silence exhaustive-deps
Prettier	inconsistent formatting	<code>npx prettier --write .;</code> format on save	add <code>eslint-config-prettier</code> so ESLint does not fight it
React DevTools	wrong props or state, unexpected re-renders	browser extension: Components and Profiler tabs	select a component, then edit its state live
Elements tab	which classes and styles really apply	built into the browser (F12)	check hashed CSS Module names and the cascade
Vitest	regressions in behaviour	<code>npx vitest (watch), npx vitest run (once)</code>	run it in watch mode while you work

Example. ESLint underlines an effect: "React Hook `useEffect` has a missing dependency: `'taskId'`. Either include it or remove the dependency array." Adding `taskId` to the array fixes a real bug: without it, the effect kept loading the first task after the user opened another one.

Common mistake. Silencing a warning instead of understanding it: `// eslint-disable-next-line react-hooks/exhaustive-deps` above the dependency array. The warning disappears, the stale-value bug it was pointing at stays. Fix the effect (move the function inside it, add the dependency, or use an updater function).

Finding elements

Testing Library: test what the user sees

React Testing Library renders a component into a simulated page and finds things the way a person does, by role, label and text: the more a test resembles real use, the more confidence it gives.

Setting it up

1. Install the tools as development dependencies: `npm install -D vitest jsdom @testing-library/react @testing-library/dom @testing-library/user-event @testing-library/jest-dom`. `@testing-library/dom` is listed because current versions of the React package expect you to install it yourself.
2. In `vite.config.js`, set environment: `'jsdom'` (tests get a document), `globals: true` (describe, it and expect without imports, and automatic clean-up after each test) and a `setupFiles` entry.
3. In that setup file, load the DOM matchers once: `import '@testing-library/jest-dom/vitest';`.

```
// vite.config.js
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
  plugins: [react()],
  test: {
    environment: 'jsdom',
    globals: true,
    setupFiles: ['./src/setupTests.js'],
  },
});
```

```
// src/setupTests.js
import '@testing-library/jest-dom/vitest';
```

render(<Counter />)	Mounts the component with the real React into document.body, provided by jsdom. After each test it is unmounted, so tests do not leak into each other.
screen	Holds the queries for the whole page: screen.getByRole(...).
userEvent	Performs realistic clicks and typing (see Simulating the user).
jest-dom matchers	Assertions about the DOM: toBeInTheDocument, toHaveTextContent, toBeDisabled, toHaveValue.

```
import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import Counter from './Counter';

test('the count goes up when the user clicks Increment', async () => {
  const user = userEvent.setup();
  // Arrange
  render(<Counter />);

  await user.click(screen.getByRole('button', { name: 'Increment' })); // Act

  // Assert
  expect(screen.getByText('Count: 1')).toBeInTheDocument();
});
```

Example. Read the test aloud: "render the counter; click the button named Increment; the text Count: 1 is on the page". It mentions nothing a user could not see. If Counter is rewritten with useReducer, split into two components and moved to CSS Modules, the test stays green as long as the screen behaves the same.

Common mistake. Looking for the state: wanting to check that count is 1, or reaching for a library that exposes component internals. State is a means; the behaviour is what the screen shows. Assert on the text, the attributes and the callbacks, and the test keeps working when the state is reorganised.

Finding elements: by role, label and text

Queries find elements in the rendered page; prefer the ones a person would use, role and name first and test ids last.

1. `getByRole`, with name
 2. `getByLabelText`, for form fields
 3. `getByPlaceholderText`
 4. `getByText`, for non-interactive text
 5. `getByDisplayValue`
 6. `getByAltText`, `getByTitle`
 7. `getByTestId`, only as a last resort
- **Options:** `screen.getByRole('button', { name: 'Add task' })`; the name can be a regular expression, `{ name: /add/i }`; headings take `{ level: 2 }`; toggles and checkboxes take `{ pressed: true }` or `{ checked: true }`.
 - **Read the error:** when nothing matches, it lists every role and name on the page, which usually shows the typo or the missing label.
 - **Text is exact** after trimming: `getByText('Count: 1')` does not match "Count: 10". Pass `{ exact: false }` for a case-insensitive substring, or a regular expression.
 - **Each element's own text:** `<p>Count: <strong>1</strong></p>` is two pieces, so `getByText('Count: 1')` finds neither.
 - **Inside one part of the page:** `within(element).getByRole(...)`.

Implicit roles of common elements

Element	Role	Accessible name from
<button>, <input type="submit">	button	its text, or value
<a href>	link	its text
<h1>...<h6>	heading (level 1–6)	its text
<input> (text, email), <textarea>	textbox	its <label> (or aria-label)
<input type="checkbox"> / "radio"	checkbox / radio	its <label>
<select>	combobox	its <label>
, /	list / listitem	aria-label (optional)
	img	alt
<nav>, <main>, <form>	navigation, main, form	aria-label (optional)
role="alert" / role="status"	alert / status	its text

Example. For a task form: `screen.getByLabelText('Task title')` finds the input through its `<label>`; `screen.getByRole('button', { name: 'Add task' })` finds the submit button; after an empty submit, `screen.getByRole('alert')` finds the error message. None of them depends on a class, an id or the order of the elements.

Common mistake. Switching to `getByPlaceholderText` or a test id because `getByRole('textbox', { name: 'Title' })` fails. The failure is telling you something: the input has only a placeholder and **no label**, so a screen reader announces it as "edit text" with no name. Fix the markup with a `<label htmlFor>`, and the role query passes.

getBy, queryBy, findBy: present, absent, later

Every query comes in three flavours: `getBy...` returns the element or **throws**, `queryBy...` returns **null** when it is absent, and `findBy...` returns a **promise** that resolves when it appears.

- `findBy...` **is** `getBy...` **retried** every 50 ms for up to 1 s (change it with `findByText('...', { timeout: 3000 })`). `await` it in an `async` test; while it waits, React keeps rendering, so data from a `fetch` has time to appear.
- Absence:** `expect(screen.queryByRole('alert')).not.toBeInTheDocument()` (or `.toBeNull()`).
- Waiting for something to go:** `await waitForElementToBeRemoved(() => screen.queryByText('Loading...'))`, or any assertion inside `await waitFor(() => ...)`.

Which query, for which question

Query	0 matches	1 match	More than 1	Waits?	Use it for
<code>getBy...</code>	throws	the element	throws	no	it is there now
<code>queryBy...</code>	null	the element	throws	no	it is not there
<code>findBy...</code>	rejects after 1 s	the element	rejects	yes: <code>await</code>	it will appear
<code>getAllBy...</code>	throws	[el]	[el, el...]	no	several are there now
<code>queryAllBy...</code>	[]	[el]	[el, el...]	no	counting, possibly zero
<code>findAllBy...</code>	rejects after 1 s	[el]	[el, el...]	yes: <code>await</code>	several will appear

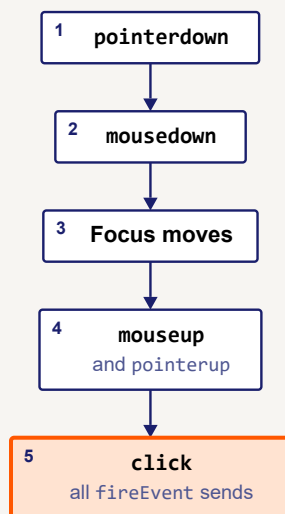
Example. A form with validation: before submitting, `expect(screen.queryByRole('alert')).not.toBeInTheDocument();` after an empty submit, `expect(screen.getByRole('alert')).toHaveTextContent('Title is required')`. A list that loads: `expect(await screen.findByText('Buy milk')).toBeInTheDocument(),` then `expect(screen.getAllByRole('listitem')).toHaveLength(5).`

Common mistake. Checking absence with `getBy`: `expect(screen.getByText('Loading...')).not.toBeInTheDocument()`. `getBy` throws **before** `expect` runs, so the test fails with "Unable to find an element" precisely when the loading message is correctly gone. Absence needs `queryBy`.

Interactions and async UI

Simulating the user: `userEvent` and mock callbacks

`@testing-library/user-event` **simulates complete interactions** (`await user.click(button)`, `await user.type(input, 'Buy milk')`), and `vi.fn()` **checks what a component tells its parent through callback props**.



One user click is five events; `fireEvent` sends only the last.

`const user = userEvent.setup()`

At the start of the test. Every action returns a promise: `await` each one.

`user.type(e1, 'Call Ana{Enter}')`

Types, then presses Enter. `user.clear(e1)` empties a field.

`user.selectOptions(select, 'done')`

Picks an option.

`user.keyboard('{Escape}'), user.tab()`

Presses keys on the focused element; moves the focus.

```

import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import { vi } from 'vitest';
import TodoForm from './TodoForm';

test('adds a trimmed task and clears the field', async () => {
  const user = userEvent.setup();
  const onAdd = vi.fn();
  render(<TodoForm onAdd={onAdd} />);

  const field = screen.getByLabelText('Task title');
  await user.type(field, ' Buy milk ');
  await user.click(screen.getByRole('button', { name: 'Add task' }));

  expect(onAdd).toHaveBeenCalledWith('Buy milk');
  expect(field).toHaveValue('');
});

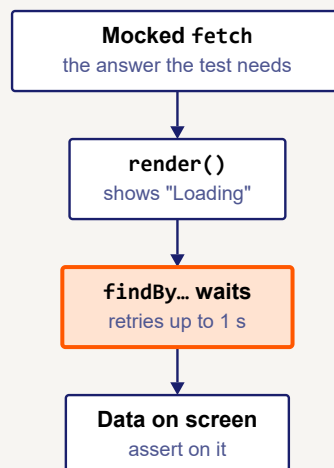
```

Example. The test types into the field found by its label, clicks the button found by its role and name, and checks the two visible effects of a successful submit: the parent received the trimmed title, and the field is empty again. If the form forgot to trim, the failure reads expected "spy" to be called with arguments: ['Buy milk'] and lists the actual call, [' Buy milk '].

Common mistake. Forgetting `await`: `user.click(button); expect(onAdd).toHaveBeenCalled();`. The click is still in progress when the assertion runs, so the test fails, or worse, passes by luck and fails later on a slower machine. Every `user....` call returns a promise: `await` it (the ESLint rule `testing-library/await-async-events` reminds you).

Testing async UI: waiting and mocking fetch

A component that loads data shows "Loading..." first and the data later, so its tests must **wait** (`await screen.findBy...`) and **control the network** with a mocked fetch.



The test controls the answer, then waits exactly as long as needed.

Making fetch answer, one test at a time

The test needs	Code
Data	<code>vi.spyOn(globalThis, 'fetch').mockResolvedValue(Response.json([{ id: 1, title: 'Buy milk' }]))</code>
A server error	<code>.mockResolvedValue(new Response(null, { status: 500 })))</code>
No connection	<code>.mockRejectedValue(new TypeError('Failed to fetch'))</code>
The request itself	<code>expect(fetch).toHaveBeenCalledTimes(1)</code>
Clean-up	<code>vi.restoreAllMocks()</code> in <code>afterEach</code> (or <code>restoreMocks: true</code> in the config)

Mock the API client

If components call one API client module, mock it with `vi.mock('./api/client')` and return plain data.

Mock the network: MSW

Mock Service Worker answers requests at the network level, so the component's real fetch code, URL and headers included, runs in the test. In the test runner here, a small fake API plays that role for `/api/tasks`.

```

import { render, screen } from '@testing-library/react';
import { afterEach, vi } from 'vitest';
import TaskList from './TaskList';

afterEach(() => vi.restoreAllMocks());

test('shows the tasks once they arrive', async () => {
  vi.spyOn(globalThis, 'fetch').mockResolvedValue(
    Response.json([{ id: 1, title: 'Buy milk', done: false }])
  );
  render(<TaskList />);
  expect(screen.getByRole('status')).toHaveTextContent('Loading');
  expect(await screen.findByText('Buy milk')).toBeInTheDocument();
});

test('tells the user when the server fails', async () => {
  vi.spyOn(globalThis, 'fetch').mockResolvedValue(new Response(null, { status: 500 }));
  render(<TaskList />);
  expect(await screen.findByRole('alert')).toHaveTextContent('Could not load tasks');
});

```

Example. The first test checks both moments: right after render the status says "Loading" (a plain `getBy`, it is there now), and `findByText` then waits for the mocked answer to be rendered. Change `findByText` to `getByText` and the test fails, because the promise from `fetch` has not resolved yet when the line runs.

Common mistake. Waiting with a fixed sleep: `await new Promise((r) => setTimeout(r, 2000))`. Every test now takes two seconds, and it still fails on a slow machine where the data needs 2.1. `findBy...` and `waitFor` wait **exactly** as long as needed, up to a timeout, and report what was on the page if it never came.

What not to test: implementation details

An **implementation detail** is anything the user cannot see or use (state names, internal functions, class names, the structure of the markup), and tests that check them fail in both directions.

False alarm A class moves from `.counter__inc` to a CSS Module; nothing changed for users, but a test using `container.querySelector('.counter__inc')` turns red. After a few of these, people stop trusting red tests.

False confidence A toggle test checks that the class `toggle--on` is added, but the code forgot `aria-pressed`: a screen-reader user cannot tell the button is on, and the test is green.

Avoid

- Finding elements by class, id or position (`container.querySelector, firstChild`).
- Asserting on class names or inline styles to mean a state; reading `.textContent` or `.className` yourself.
- Test ids when a role or a label exists.
- Snapshots of whole components: huge diffs that get approved without reading.
- Testing React itself ("setState re-renders").

From brittle to robust

Brittle	Robust
<code>container.querySelector('.btn-primary')</code>	<code>screen.getByRole('button', { name: 'Save' })</code>
<code>getByTestId('title-input')</code>	<code>getByLabelText('Task title')</code>
<code>expect(p.textContent).toBe('Count: 1')</code>	<code>expect(screen.getByText('Count: 1')).toBeInTheDocument()</code>
<code>expect(button.className).toContain('toggle--on')</code>	<code>expect(button).toHaveAttribute('aria-pressed', 'true')</code>
<code>expect(link).toHaveClass('active')</code>	<code>expect(link).toHaveAttribute('aria-current', 'page')</code>
a snapshot of the whole form	two or three assertions about what the test title promises

Example. The toggle story in code: the brittle test `expect(container.firstChild.className).toBe('toggle toggle--on')` breaks when the styles move to a CSS Module (the class becomes `toggle on`) and still passes when `aria-pressed` is stuck at `false`. The robust one, `expect(screen.getByRole('button', { name: 'Mute', pressed: true })).toBeInTheDocument()`, survives the refactor and catches the bug.

Common mistake. "More assertions make a better test": checking every class, every attribute and the exact markup of a component. Each one is a reason for the test to fail on a harmless change, and none adds confidence about behaviour. Assert what the test's name promises, and stop.

A first component test, end to end

A first component test from start to finish: set the tools up once, write the test next to the component, run it, then make it fail on purpose and read the failure.

1. **Set up once:** install Vitest, jsdom and Testing Library, and configure them (see [Testing Library](#)).
2. **Write the test** next to the component: `src/components/ToDoForm.test.jsx`.
3. **Run** `npm test` and watch it go green.
4. **Break it on purpose** (change the expected text) and read the failure: the message, the page Testing Library printed, the line. Reading that output quickly is half of testing.
5. **Fix it** and keep watch mode running while you work.

```
// src/components/ToDoForm.test.jsx
import { render, screen } from '@testing-library/react';
```

```
import userEvent from '@testing-library/user-event';
import TodoForm from './TodoForm';

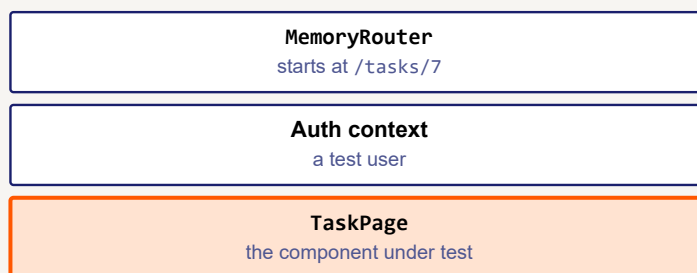
describe('TodoForm', () => {
  it('shows an error for an empty title', async () => {
    const user = userEvent.setup();
    const onAdd = vi.fn();
    render(<TodoForm onAdd={onAdd} />);
    await user.click(screen.getByRole('button', { name: 'Add task' }));
    expect(screen.getByRole('alert')).toHaveTextContent('Title is required');
    expect(onAdd).not.toHaveBeenCalled();
  });
});
```

Example. `npm test` prints ✓ `src/components/TodoForm.test.jsx` (1 test). Change the label query to `getByLabelText('Title')` in another test and the failure reads "Unable to find a label with the text of: Title", followed by the rendered HTML, where you can see the label actually says "Task title".

Common mistake. Forgetting environment: 'jsdom'. Vitest runs tests in plain Node by default, so the first render fails with "ReferenceError: document is not defined". And without the setup file, `toBeInTheDocument` fails with "Invalid Chai property": the jest-dom matchers were never loaded.

Components that need a provider or a router: render with a wrapper

A component that reads a context or uses router hooks only works inside its provider or router, so a test renders it with one: `render(ui, { wrapper })`.



The wrapper rebuilds, around one component, what the app puts above it.

`render(ui, { wrapper: Wrapper })`

Wrapper is a component that receives children and renders the providers around them. `rerender` keeps the same wrapper.

`<MemoryRouter initialEntries={['/tasks/7']}>`

A router that keeps its history in memory instead of the address bar, starting at the URL the test chooses (see [Setting up a router](#)).

A `renderWithProviders` helper

One function, in a test-utilities file, that wraps every provider the app uses and accepts overrides per test (another user, another URL).

```
// src/test/renderWithProviders.jsx
import { render } from '@testing-library/react';
import { MemoryRouter } from 'react-router';
import { AuthContext } from '../auth/AuthContext';

export function renderWithProviders(ui, { user = { name: 'Ana' }, route = '/' } = {}) {
  function Wrapper({ children }) {
    return (
      <MemoryRouter initialEntries={[route]}>
        <AuthContext value={{ user }}>{children}</AuthContext>
      </MemoryRouter>
    );
  }
  return render(ui, { wrapper: Wrapper });
}
```

```
}

// src/pages/TaskPage.test.jsx
import { screen } from '@testing-library/react';
import { Routes, Route } from 'react-router';
import { renderWithProviders } from '../test/renderWithProviders';
import TaskPage from './TaskPage';

test('shows the task in the URL to a logged-in user', async () => {
  renderWithProviders(
    <Routes><Route path="/tasks/:id" element={<TaskPage />} /></Routes>,
    { route: '/tasks/7' },
  );
  expect(await screen.findByRole('heading', { name: /task 7/i })).toBeInTheDocument();
});
```

Example. A settings page shows "Log in to see your settings" to guests. `renderWithProviders(<Settings />, { user: null })` checks that message; `renderWithProviders(<Settings />)` checks that Ana's email is shown. Same component, two contexts, two tests, and neither needs a real log-in.

Common mistake. Rendering a page that calls `useNavigate` or `useParams` with a bare `render(<TaskPage />)`. The test fails before any assertion with "useNavigate() may be used only in the context of a <Router> component". The component is fine: the test forgot what the app puts above it. Wrap it in a `MemoryRouter`.